



Lista 3 — Estado, APIs e Persistência

Aulas 19 a 27

Desenvolvimento Móvel

Exercícios sobre gerenciamento de estado (setState, Provider, Riverpod), consumo de APIs REST e persistência de dados. Cenários de aplicações conectadas e responsivas.

Exercício 1

Cenário: App de clima que consulta API externa (OpenWeatherMap) e exibe previsão.

1. Crie um modelo WeatherModel com: cityName, temperature, humidity, windSpeed, iconUrl, description.
2. Use o pacote http para buscar dados da API. Crie um WeatherService com método Future<WeatherModel> getWeather(String city).
3. Gerencie estado com ChangeNotifier + Provider: WeatherNotifier com estado {weather, loading, error}.
4. Crie a UI: SearchBar, Card com clima atual, ListView com previsão para 5 dias. Exiba loading (CircularProgressIndicator) e erros (SnackBar).
5. Extra: adicione geolocalização com geolocator package para clima automático.

Exercício 2

Cenário: App de tarefas com persistência local usando SQLite.

1. Use o pacote sqflite: crie tabela tarefas (id INTEGER PK, titulo TEXT, descricao TEXT, concluida INTEGER, dataCriacao TEXT).
2. Implemente um TaskDao com métodos: insert, getAll, getById, update, delete.
3. Crie um TaskNotifier (ChangeNotifier) que carrega tarefas ao iniciar, notifica listeners a cada alteração.
4. UI: ListView com Checkbox para concluir, Dismissible para deletar, FloatingActionButton para adicionar.

Exercício 3

Cenário: App de filmes com busca, favoritos e tela de detalhes.

1. Use a API TMDb (The Movie Database, gratuita) para buscar filmes. Crie `MovieService` com: `searchMovies`, `getPopular`, `getMovieDetails`.
2. Use `Provider` ou `Riverpod` para gerenciar estado: `movieList`, `favorites`, `loading`, `error`.
3. Implemente favoritos: ícone de coração na lista e na tela de detalhes, persista com `SharedPreferences` (lista de IDs).
4. Tela de detalhes: pôster grande, título, ano, sinopse, nota (5 estrelas), gêneros, elenco principal.
5. Teste com mock: substitua a API real por dados mockados durante o desenvolvimento usando injeção de dependência.

Exercício 4

Cenário: App de chat simples com State management centralizado (Bloc ou Riverpod).

1. Modele `Message`: `{id, userId, userName, text, timestamp}`. Use sealed class para estados: `ChatInitial`, `ChatLoading`, `ChatLoaded(List<Message>)`, `ChatError(String)`.
2. Implemente um `ChatBloc` ou `ChatNotifier` que gerencia o estado e expõe métodos: `sendMessage`, `loadMessages`.
3. UI: mensagens alinhadas à direita (usuário) e esquerda (outros), `TextField` com envio ao pressionar `Enter`, botão de enviar.
4. Pesquise sobre injeção de dependência no Flutter: `get_it` vs `Riverpod`. Qual a vantagem de desacoplar serviços das telas?

Requisitos

Flutter SDK. Pacotes: `provider`, `http`, `sqlite`, `path`, `shared_preferences`, `geolocator`. Conta TMDb (API key gratuita).